MIT AI2 204

IoT with MIT App Inventor

Fundamental

X. Tang

Get started with teachable machine

This project has two parts to it: first you build a model on Teachable Machine, and then you use that model in a game you'll build on your computer.

Teachable Machine is a tool made by Google that creates machine learning models in your browser. You'll create a model to recognise images, but it can also create models that recognize sounds, or body poses.

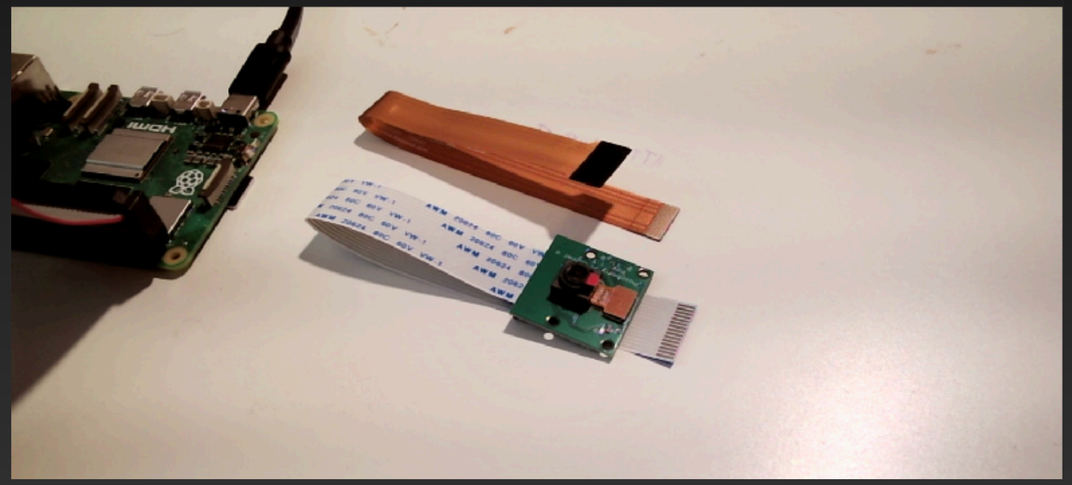
Get started with teachable machine

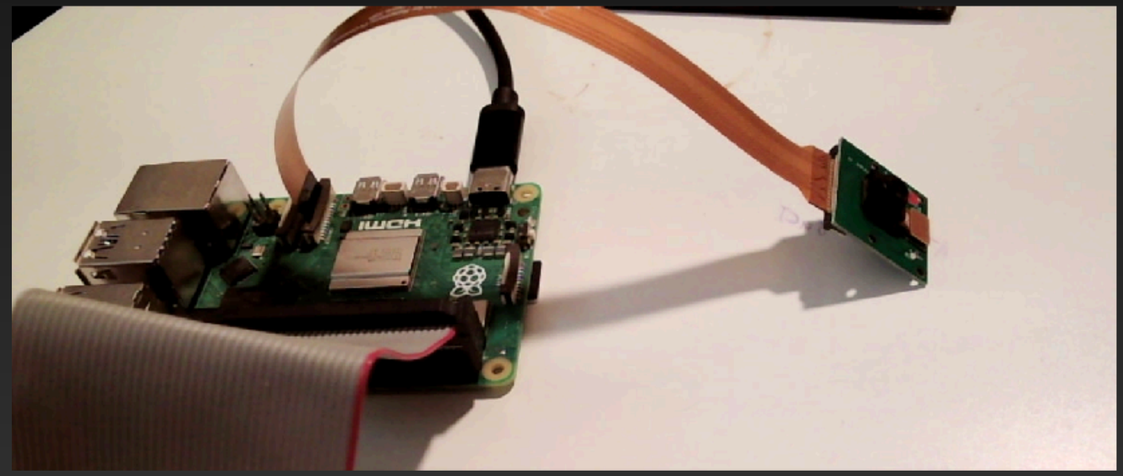
Setup your camera(purchased), or use USB connected Camera

Get started with teachable machine

Setup your camera(purchased)

Pi 5 needs to use the adapter connector with bronze color, smaller side connect to Pi 5.



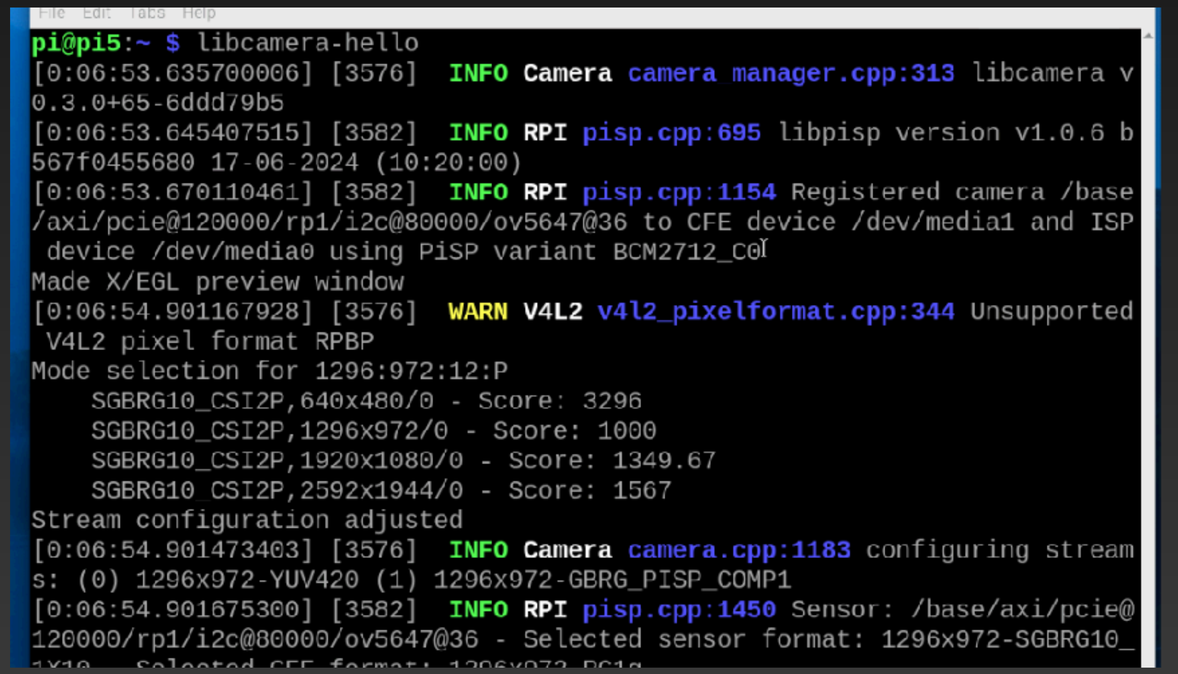


Get started with teachable machine

Open Pi 5 and test the connection of the camera.

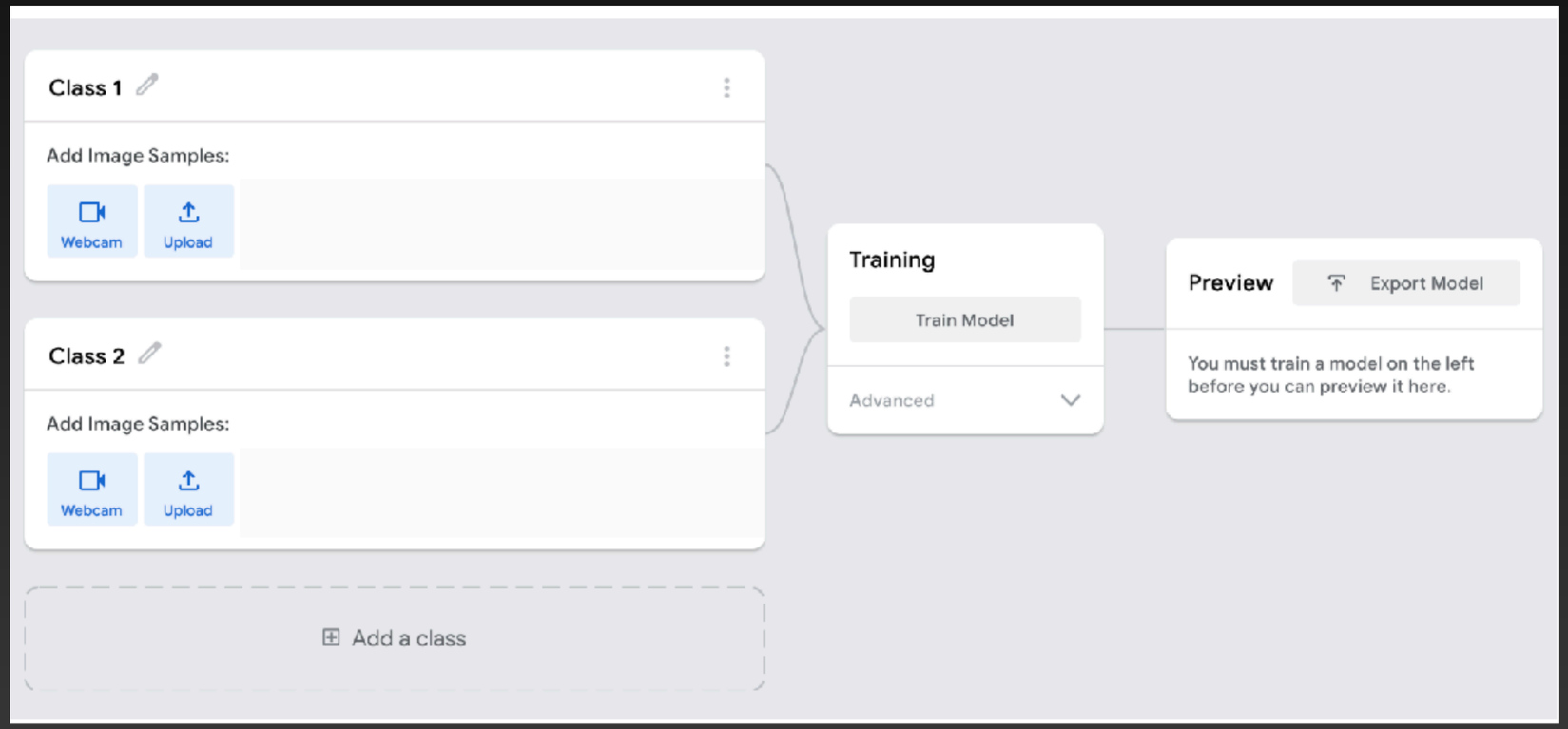
In the terminal, type command: libcamera-hello

System detect camera and will show message below:



Get started with teachable machine

https://teachablemachine.withgoogle.com/train/image



Get started with teachable machine

Open Thonny Python to run scripts and capture pictures using below code:

ImageCapture.py

```python
from picamera2 import Picamera2, Preview
from time import sleep

picam2 = Picamera2()
picam2.start_preview(Preview.QTGL)
picam2.start()
sleep(2)

for i in range(15):
    picam2.capture_file(f"rockimage_{i:04d}.jpg")
    sleep(1)

picam2.stop_preview()
picam2.stop()
```

from picamera2 import Picamera2, Preview
from time import sleep

picam2 = Picamera2()
picam2.start_preview(Preview.QTGL)
picam2.start()
sleep(2)

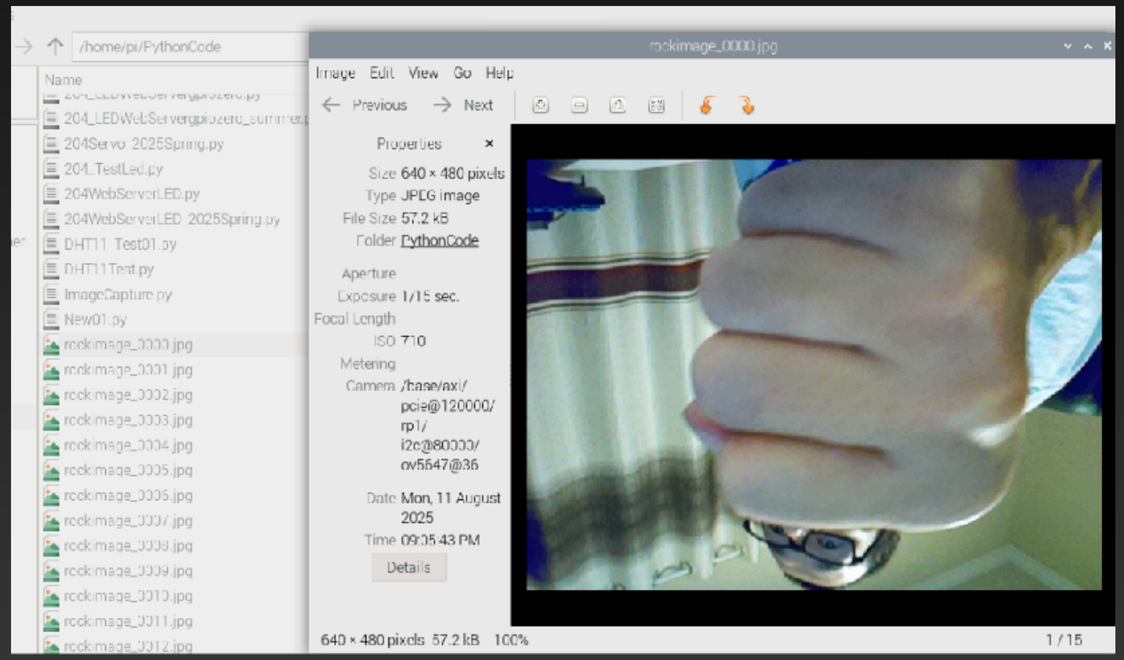
for i in range(15):
picam2.capture_file(f"sissorsimage_{i:04d}.jpg")
sleep(1)

picam2.stop_preview()
picam2.stop()

Get started with teachable machine

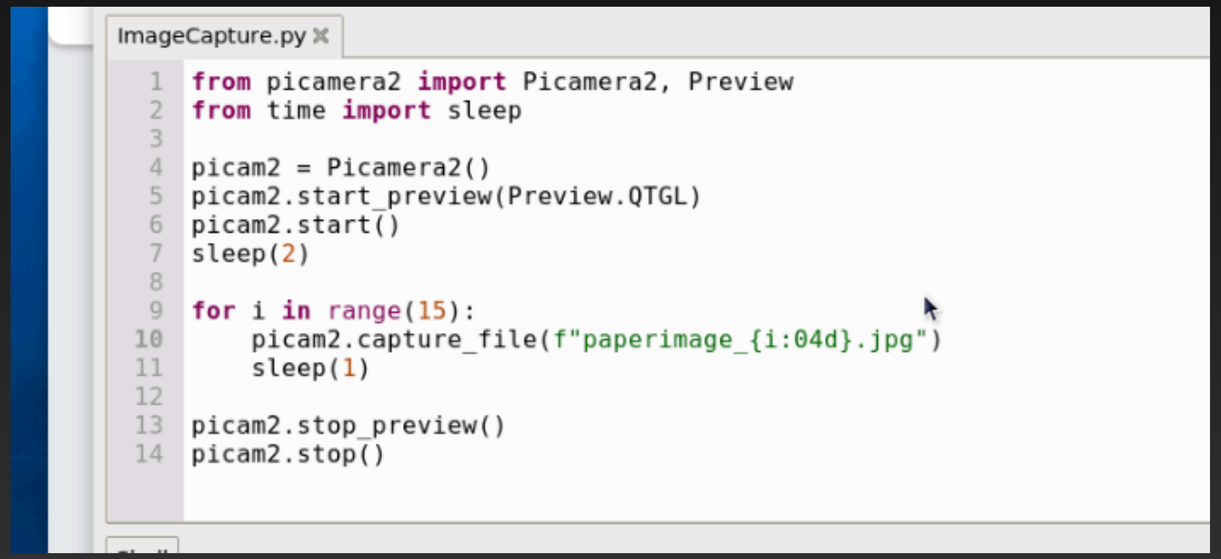
You will find the pictures taken and stored in the folder where Thonny Python stores scripts

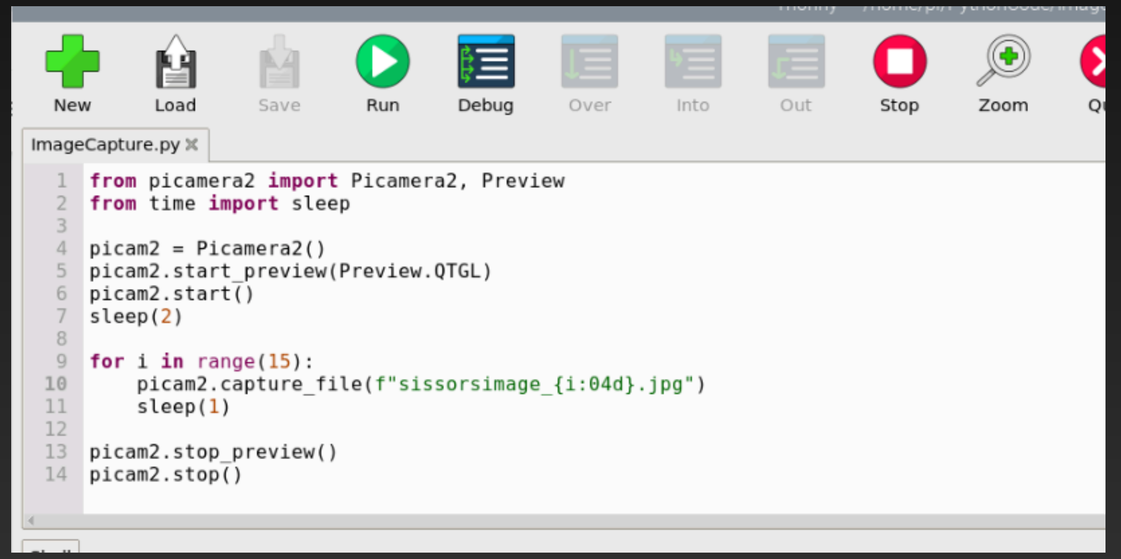
```
pi@pi5:~ $ ls
Adafruit_Python_DHT  Downloads              Music          quarantine_zone
Bookshelf            Freenove_01            myenv          rps_by_hand
Desktop              Freenove_Kit           Pictures       Templates
DHT11                Freenove_Kit_25summer  Public         Videos
dht_test             GPIO-Music-Box         PythonCode
dht_test01           hi.py                  python_games
Documents            mu_code                quarantine
pi@pi5:~ $ cd PythonCode
pi@pi5:~/PythonCode $ ls
204_BuzzerTest_2025Spring.py        204WebServerLED.py   rockimage_0006.jpg
204_DHT11_Test.py                   DHT11_Test01.py      rockimage_0007.jpg
204_DistanceTest_2025Spring.py      DHT11Test.py         rockimage_0008.jpg
204_FlashingLED.py                  ImageCapture.py      rockimage_0009.jpg
204_LED_BCM.py                      New01.py             rockimage_0010.jpg
204_LEDgpiozero.py                  rockimage_0000.jpg   rockimage_0011.jpg
204_LEDWebServergpiozero.py         rockimage_0001.jpg   rockimage_0012.jpg
204_LEDWebServergpiozero_summer.py  rockimage_0002.jpg   rockimage_0013.jpg
204Servo_2025Spring.py              rockimage_0003.jpg   rockimage_0014.jpg
204_TestLed.py                      rockimage_0004.jpg
204WebServerLED_2025Spring.py       rockimage_0005.jpg
pi@pi5:~/PythonCode $
```



Get started with teachable machine

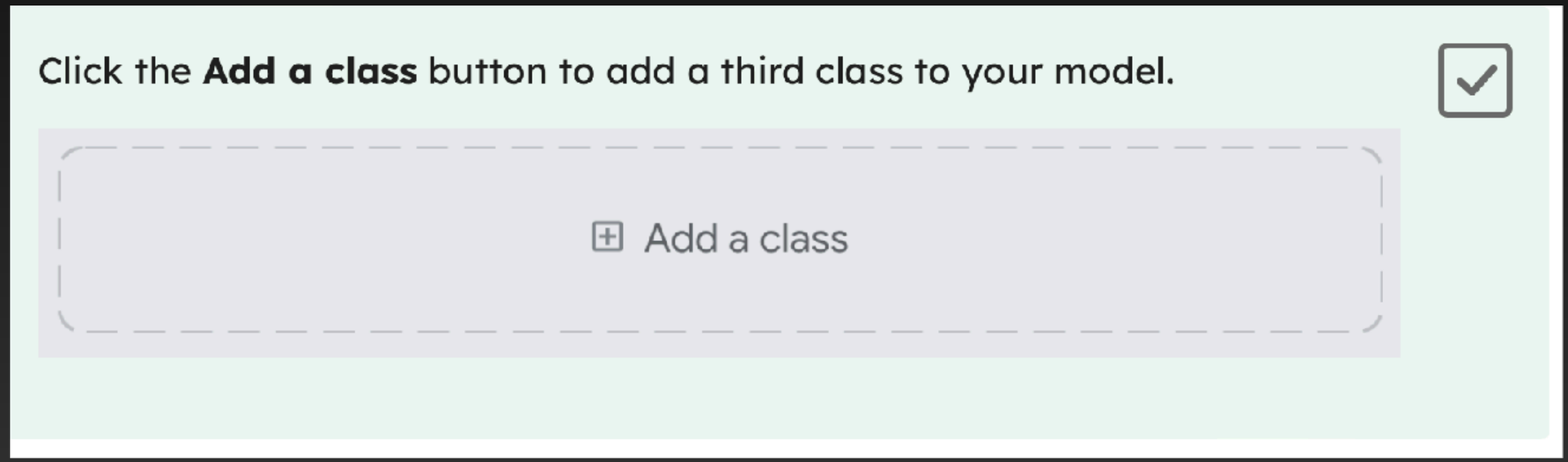
Repeat the same to capture paper and scissors pictures





Get started with teachable machine

Back to Google site. This is an untrained model, with two classes. Of course, since you need to recognize ‘rock’, ‘paper’, and ‘scissors’, you’re going to need three classes.



Get started with teachable machine

Next, give your classes proper labels that match what they're going to be trained to recognize.

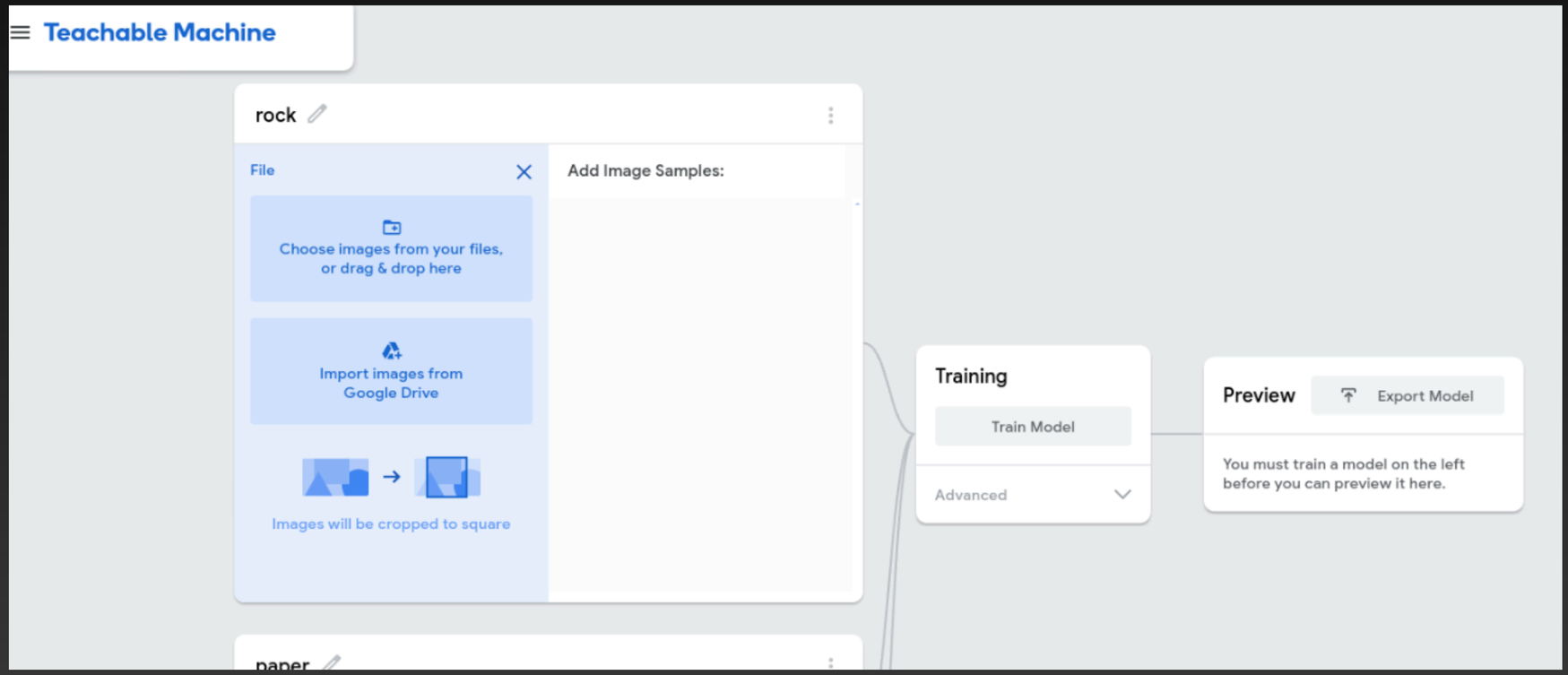
Use the pencil beside each class name to rename them as follows:

- Change 'Class 1' to 'rock'
- Change 'Class 2' to 'paper'
- Change 'Class 3' to 'scissors'

It's important to match this order, because some of the code provided to help your game run assumes that the classes are ordered like this.

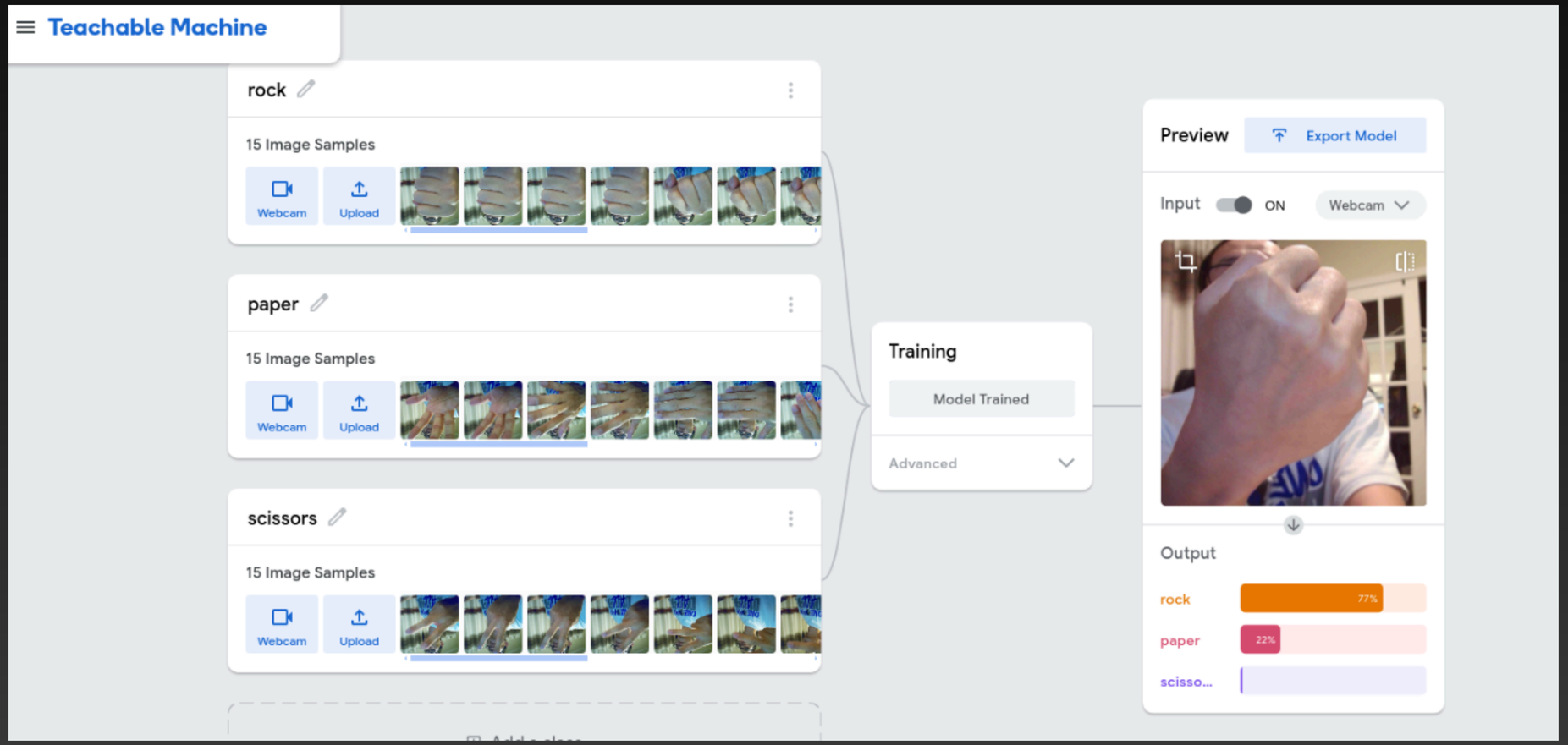
Get started with teachable machine

Next, upload the images captured using your Pi5 camera from folder where image stored.



Get started with teachable machine

Next, train the model. If you have USB connected Camera, you can do the live test.



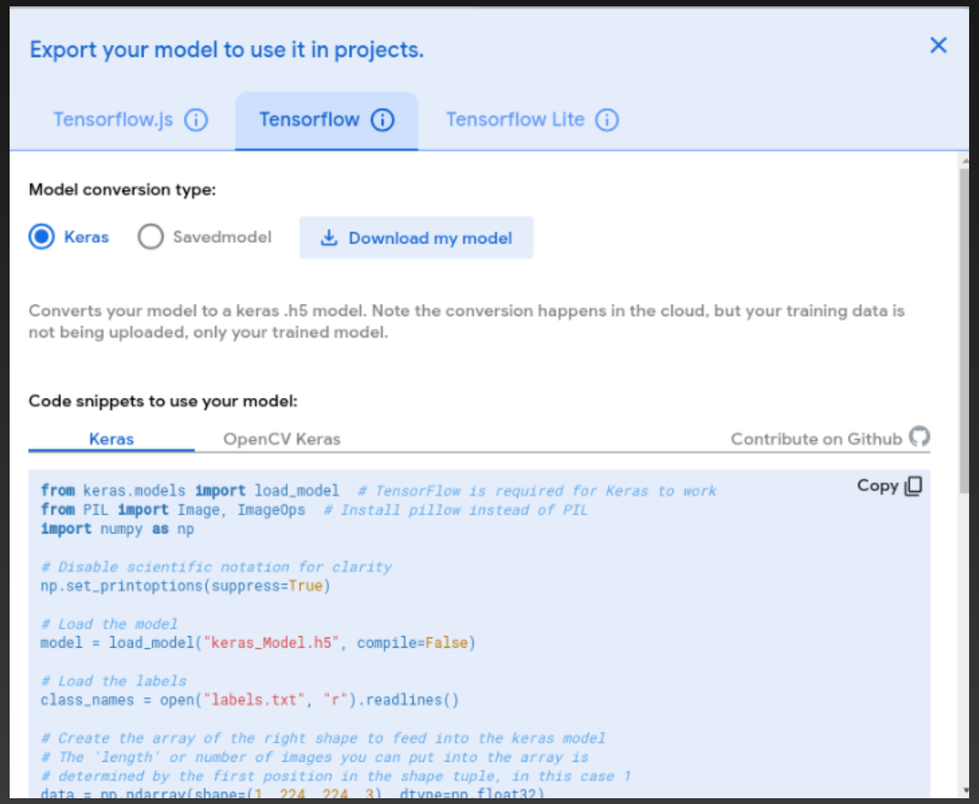
Get started with teachable machine

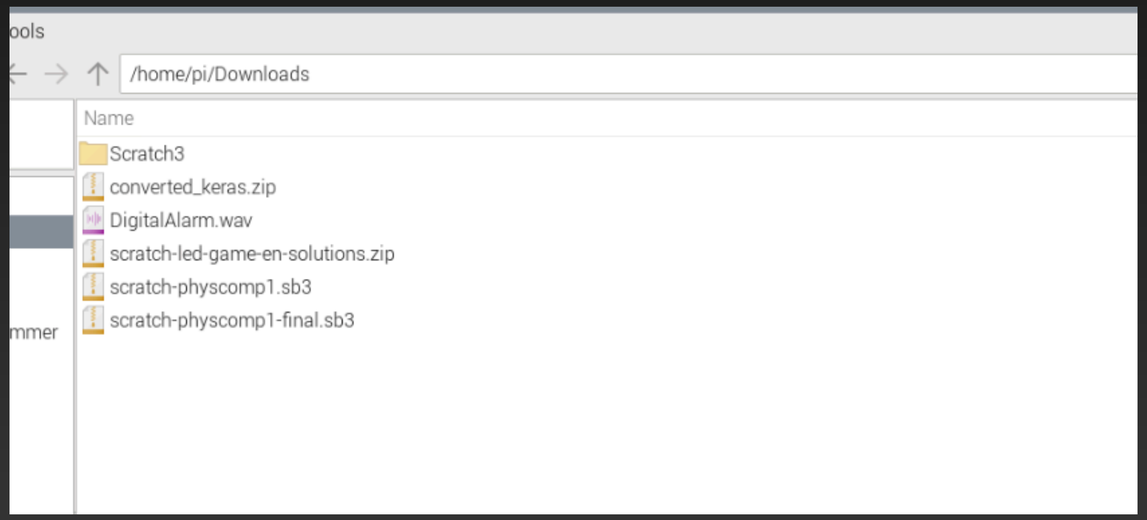
Create your model

The training will take a short time, during which you can watch Teachable Machine count the epochs it’s running through. You need to leave your browser open on the Teachable Machine tab, or the model will stop training. Once the training is complete, you’ll see a preview of the model where you can test it by making gestures at your camera and seeing how likely your model thinks that gesture is to be each of ‘rock’, ‘paper’, and ‘scissors’. The highest percentage indicates the ‘guess’ that your game would make if it saw that gesture.

Get started with teachable machine

Once your model works well enough — it will never be right all the time, but being right most of the time is pretty good — it's time to export the model so you can use it as part of your game. You will find the downloaded model in the downloads folder





Build your game

Your completed model now needs to be inserted into a desktop Python application to make a working game. A simple version of rock, paper, scissors, where the computer will use the player’s guess to cheat, has been written for you to use. You are focusing on the machine learning aspects of the project.

Build your game

For Linux (including Raspberry Pi):

Run in terminal: curl -L https://raw.githubusercontent.com/xctang/RaspberryPi_MITAI2_204_2025Summer/refs/heads/main/proj-rps-updated | sudo bash -s $USER

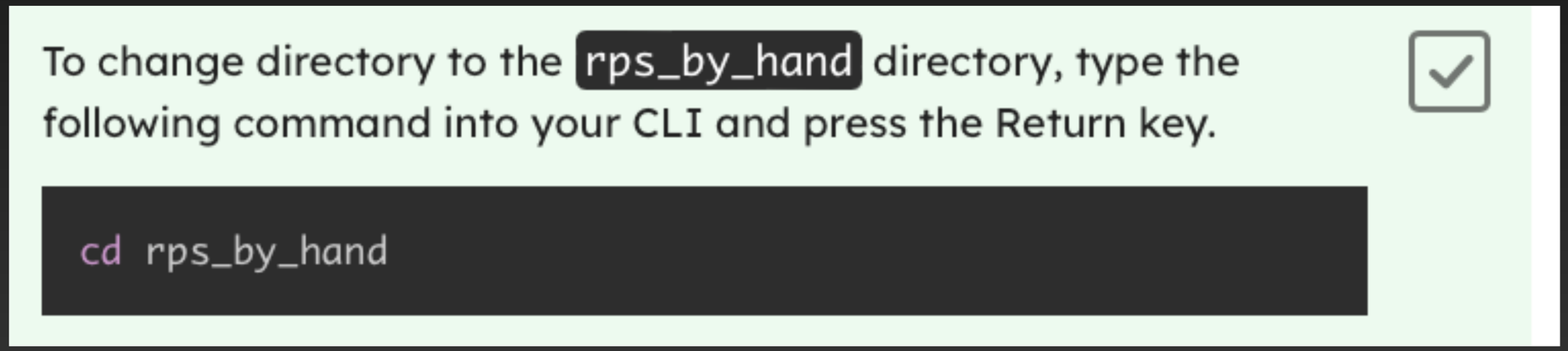
Open the CLI on your computer and type the command below in:

```
cd ~
```

Now press the Return key.

Build your game

For Linux (including Raspberry Pi): The script may take several minutes, or more, to complete the setup depending on the speed of your computer and your internet connection. Once it has finished, you will have a new directory inside your home directory, called `rps_by_hand`. This is the directory you'll work in.



Build your game

For Linux (including Raspberry Pi): In `rps_by_hand` folder. Run below command, you will see error 'No module named 'cv2'

Go back to your CLI window and type the following command to run the program. Remember it, as you'll need to run the program several times.

If you're using Windows or Linux

```
python project.py
```

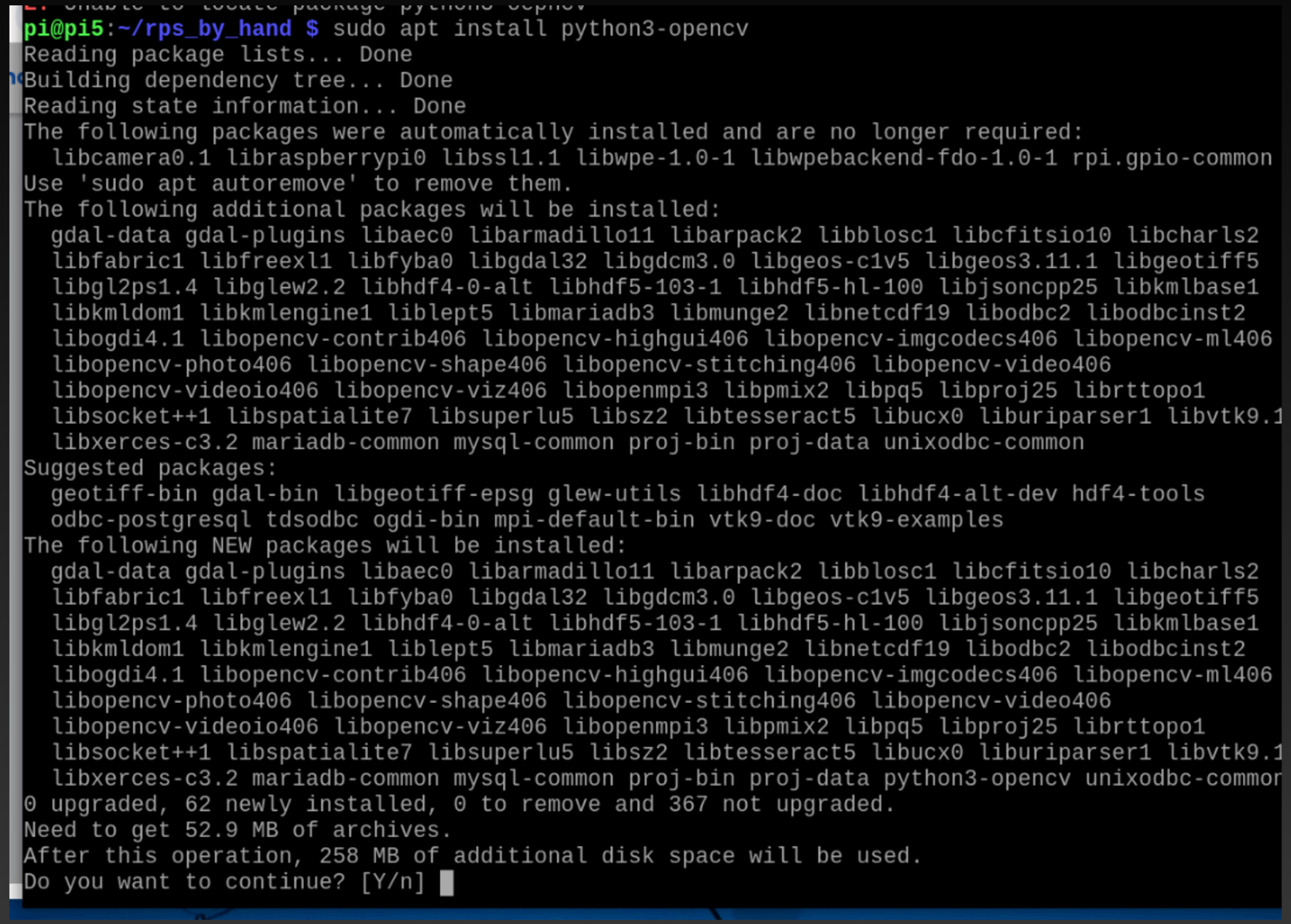
Then press the Return key to run the program.

```
note: If you believe this is a mistake, please contact your Python installation or OS distrib
rovider. You can override this, at the risk of breaking your Python installation or OS, by pa
-break-system-packages.
hint: See PEP 668 for the detailed specification.
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  2075  100  2075    0     0  10683      0 --:--:-- --:--:-- --:--:-- 10751
pi@pi5:~ $ cd rps_by_hand
pi@pi5:~/rps_by_hand $ python project.py
Traceback (most recent call last):
  File "/home/pi/rps_by_hand/project.py", line 1, in <module>
    import cv2
ModuleNotFoundError: No module named 'cv2'
pi@pi5:~/rps_by_hand $
```

Build your game

For Linux (including Raspberry Pi):

Run command: sudo apt install python3-opencv



Build your game

For Linux (including Raspberry Pi): (step1)

You will run into issues with tensorflow package not found. We need to use Python version 3.9 or 3.10

```
sudo apt update
sudo apt install -y build-essential zlib1g-dev libncurses5-dev libgdbm-dev \
libnss3-dev libssl-dev libreadline-dev libffi-dev libsqlite3-dev wget libbz2-dev
cd /usr/src
sudo wget https://www.python.org/ftp/python/3.9.18/Python-3.9.18.tgz
sudo tar xzf Python-3.9.18.tgz
cd Python-3.9.18
sudo ./configure --enable-optimizations
sudo make -j$(nproc)
sudo make altinstall
```

This installs Python3.9 as python3.9 without overwriting system python3, the installation will take a while.

Build your game

For Linux (including Raspberry Pi): (step2)

Create virtual env using Python 3.9

python3.9 -m venv ~/py39-env
source ~/py39-env/bin/activate

```
pi@pi5:/usr/src/Python-3.9.18 $ cd ~
pi@pi5:~ $ python3.9 -m venv ~/py39-env
source ~/py39-env/bin/activate
(py39-env) pi@pi5:~ $ ls
Adafruit_Python_DHT  dht_test01    Freenove_Kit_25summer  myenv       python_games     Videos
Bookshelf            Documents     GPIO-Music-Box         Pictures    quarantine
Desktop              Downloads     hi.py                  Public      quarantine_zone
DHT11                Freenove_01   mu_code                py39-env    rps_by_hand
dht_test             Freenove_Kit  Music                  PythonCode  Templates
(py39-env) pi@pi5:~ $ cd rps_by_hand
(py39-env) pi@pi5:~/rps_by_hand $ python project.py
```

Build your game

For Linux (including Raspberry Pi): no opencv found in virtual environments,

Run command: sudo apt install opencv-python

Build your game

For Linux (including Raspberry Pi): no tensorflow found in virtual environments,

Prepare in virtual env for tensorflow install:
sudo apt update
sudo apt install -y libatlas-base-dev libhdf5-dev libjpeg-dev liblapack-dev \
libblas-dev gfortran

```
(py39-env) pi@pi5:~/rps_by_hand $ python3.9 --version
Python 3.9.18
(py39-env) pi@pi5:~/rps_by_hand $ sudo apt update
sudo apt install -y libatlas-base-dev libhdf5-dev libjpeg-dev liblapack-dev \
libblas-dev gfortran
Hit:1 http://deb.debian.org/debian bookworm InRelease
Hit:2 http://deb.debian.org/debian-security bookworm-security InRelease
Hit:3 http://deb.debian.org/debian bookworm-updates InRelease
Hit:4 http://archive.raspberrypi.com/debian bookworm InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
360 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
```

Build your game

For Linux (including Raspberry Pi): no tensorflow found in virtual environments,

Tensorflow install:

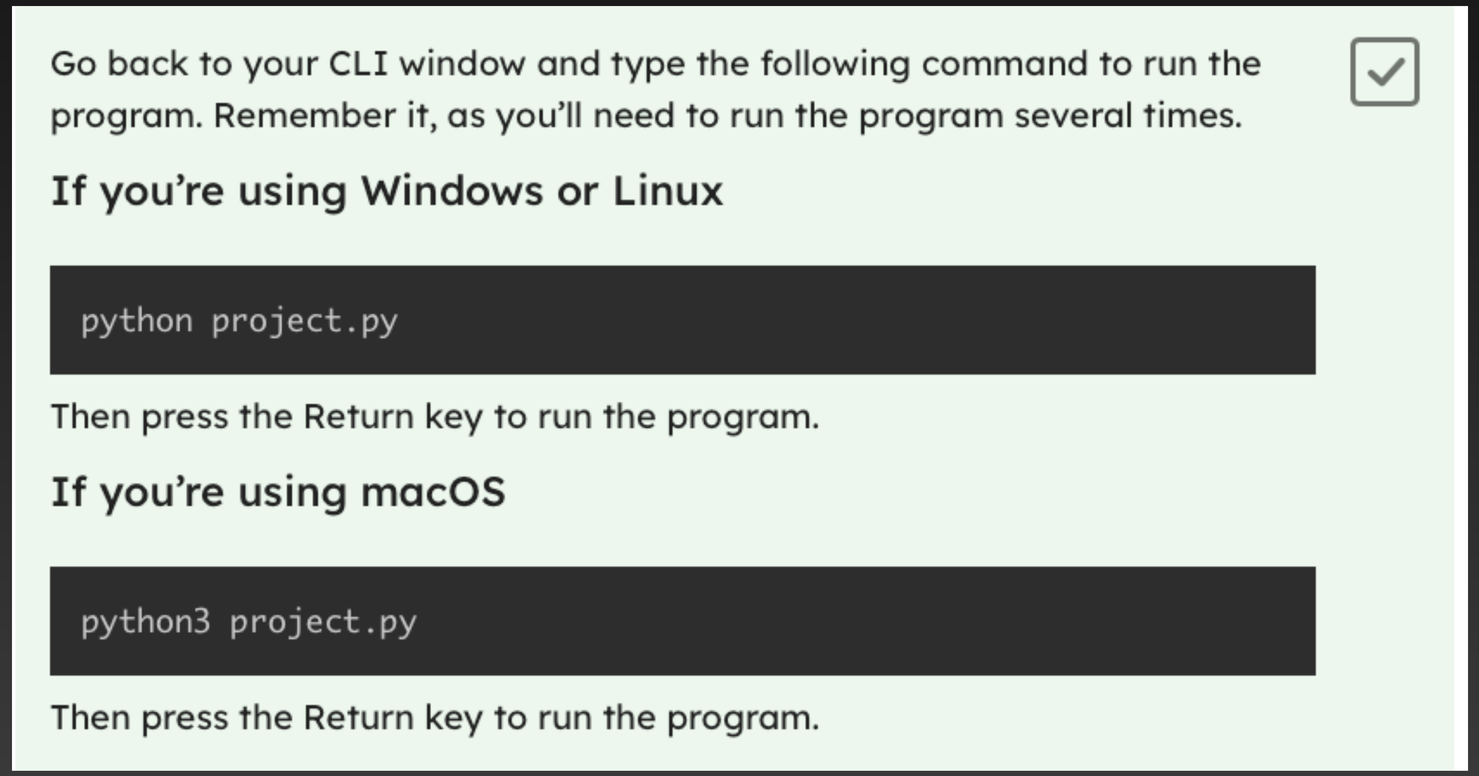
```
# Install tensorflow-io dependency for TF ≥ 2.8
git clone https://github.com/Qengineering/Tensorflow-io.git

cd Tensorflow-io

pip install tensorflow_io_gcs_filesystem-0.23.1-cp39-cp39-linux_aarch64.whl

wget -O tensorflow-2.9.1-cp39-none-linux_aarch64.whl \
    "https://drive.google.com/uc?export=download&id=1YpxNubmEL_4EgTrVMu-kYyzAbtyLis29"

pip install tensorflow-2.9.1-cp39-none-linux_aarch64.whl
```

Build your game

For Linux (including Raspberry Pi): no tensorflow found in virtual environments,

Run command:

Python project.py



Build your game

It may take a moment, but when your program runs you should see something like the image below.

Connect your model to the game

First, find the model you downloaded from Teachable Machine. It should be in a file called `converted_keras.zip`.It may take a moment, but when your program runs you should see something like the image below.

Unzip `converted_keras.zip` and open the resulting folder. Copy the `keras_model.h5` file to the project directory that contains the code for the game.

Note that the `labels.txt` file in the unzipped `converted_keras` directory contains the list of human-readable labels and the node numbers they match to on the model's output layer.

Connect your model to the game

For a larger number of categories, you would have Python read this file and automatically load them but, as there are only three categories in this model, the labels have already been included in the game code for you, in the list called `THROWS`.

Find the function `get_player_throw`. On a line above it, add the following:

```
model = tf.keras.models.load_model('keras_model.h5', compile=False)
```

Connect your model to the game

This will load your model into the game. Now you need to add a function that uses the model to get players' throws — their choice of 'rock', 'paper', or 'scissors'.

Update the `get_player_throw` function with the following code which will load the image the game has captured, then pass it to the model for a prediction. That prediction is then returned, so it can be used by the game code, instead of the default 'rock' that the game came with!

Connect your model to the game

```
def get_player_throw():
    image = tf.keras.preprocessing.image.load_img(IMG_NAME, target_size=(IMAGE_SIZE,
IMAGE_SIZE))
    image = tf.keras.preprocessing.image.img_to_array(image)
    image = np.expand_dims(image, axis=0)

    prediction_result = model.predict(image)

    best_prediction = THROWS[np.argmax(prediction_result[0])]

    return best_prediction
```

Connect your model to the game

In short:
All the lines starting with `image` work to convert the image to the right format for the model

The `prediction_result` line gets the model's prediction in the form of numbers representing confidence in different guesses

The `best_prediction` line takes the prediction with the highest value (`np.argmax` gets the index of the highest value in a list) and uses it to look up the label of that prediction in the THROWS list

Connect your model to the game

Run the program and play with it a few times! If you're not happy with the quality of its guesses, think about any differences between the training data and the inputs you give your game — maybe the light has changed, or you're wearing different clothes, etc. If you need to, you can record more training data in Teachable Machine, train your model again, and then download it and replace the model in the game with an updated version.

Build your game

For windows:

Download the starter zip file and unzip it somewhere you'll remember on your computer. If you can't think of a location, just put it on your desktop. This isn't the best place to keep things in the long term, but it's fine when you're working on them.

Next, you need to install the libraries you will use in this project. For this, you'll need to use the command line interface (CLI) — a program that controls your computer by typing text commands into a window. The command line interface is called 'command prompt' in Windows.